

# We Ain't Afraid of No File Fragmentation: Causes and Prevention of Its Performance Impact on Modern Flash SSDs

Yuhun Jun, Shinhyun Park , Jeong-Uk Kang , Sang-Hoon Kim, and Euseong Seo, Department of Semiconductor and Display Engineering, Sungkyunkwan University, Memory Business Unit, Samsung Electronics Co. Ltd. Department of Computer Science and Engineering, Sungkyunkwan University, Department of Software and Computer Engineering, Ajou University, Corresponding Author: Euseong Seo (euseong@skku.edu)

2026. 08. 25

Presentation by 김정민, 정다원

[kyrios3929@dankook.ac.kr](mailto:kyrios3929@dankook.ac.kr)

[defimedjustice1220@gmail.com](mailto:defimedjustice1220@gmail.com)

# Contents

1. Abstract
2. Introduction
3. Background
4. Motivation
5. Analysis of File Fragmentation
6. Our Approach
7. Evaluation
8. Conclusion

# 1. Abstract

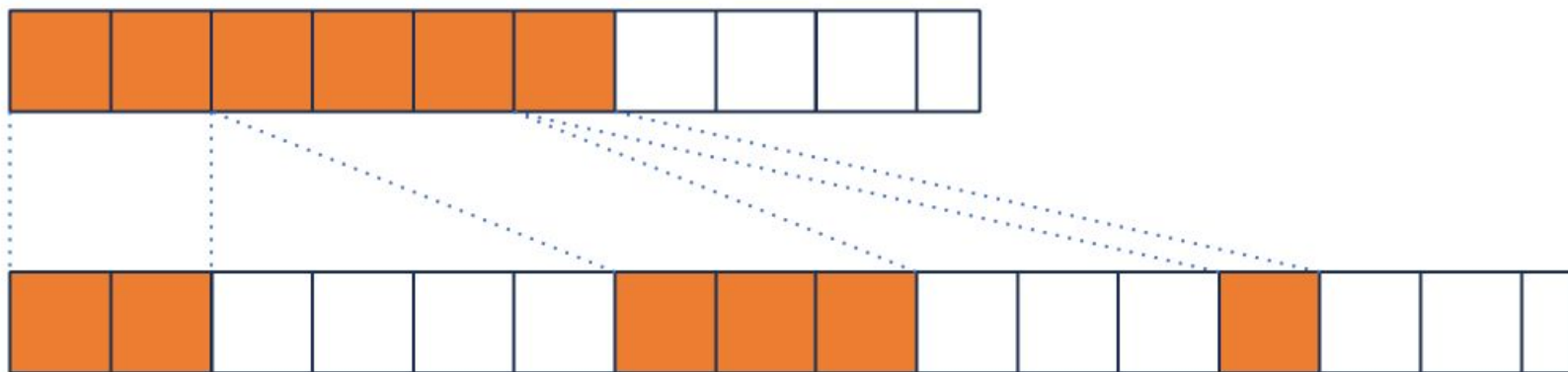
- SSD에서도 file fragmentation이 성능을 저하시킨다는 결과가 나왔고, 기존 연구에서는 그 원인을 Request Splitting이라고 생각하였다.
- 논문에서 Kernel I/O path, Host-storage interface, Flash memory access 세 계층을 분석해 본 결과, 주요 원인이 Die-level collision임을 확인하였다.
- 또한 이 문제는 파일을 overwrite할 때도 발생할 수 있다.
- 논문에서는 NVMe command extension과 Page-to-die allocation algorithm을 결합해 이 문제를 해결하고자 하였고, 그 결과 성능 저하가 40% → 3.5%로 줄어드는 모습을 보였다

## 2. Introduction

- File Fragmentation : 하나의 파일이 여러 위치에 나뉘어 저장되어 Sequential Access가 scattered access로 변환되는 현상
- SSD에서도 Fragmented File의 Read 성능이 2~5배까지 저하되는 현상이 보고됨
- 기존 연구는 성능 저하의 원인을 주로 Request Splitting으로 설명하였지만, 본 논문에서 실험을 통해 주요 원인이 Page-to-Die Mapping misalignment로 인한 Die-level Collision 증가때문임을 밝혀냄
- NVMe Command Extension + Page-to-Die Mapping Scheme 제안
  - Append: 이전 block의 다음 die에 배치
  - Overwrite: 기존 page와 같은 die에 배치
- Defragmentation 없이 fragmentation 상태를 유지하면서 성능 저하 방지

# 3. Background - File Fragmentation

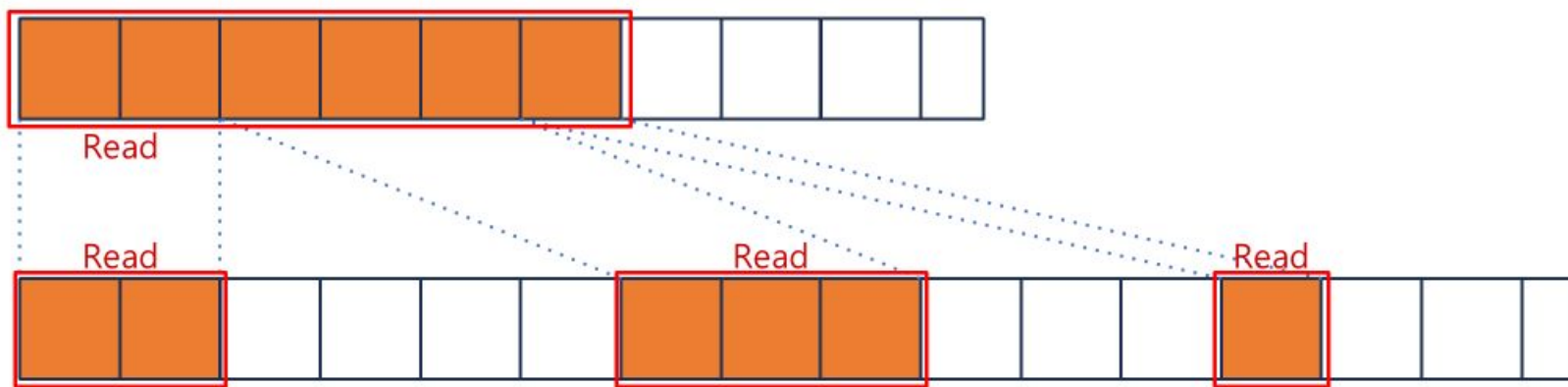
하나의 파일이 저장장치에서 여러 개의 떨어진 영역으로 나뉘어 저장되는 현상





# 3. Background - File Fragmentation

File Fragmentation이 발생하면, Disk에 접근할때, 연속된 하나의 요청이 여러 I/O 요청으로 분할될 수 있다. **(Request splitting)**



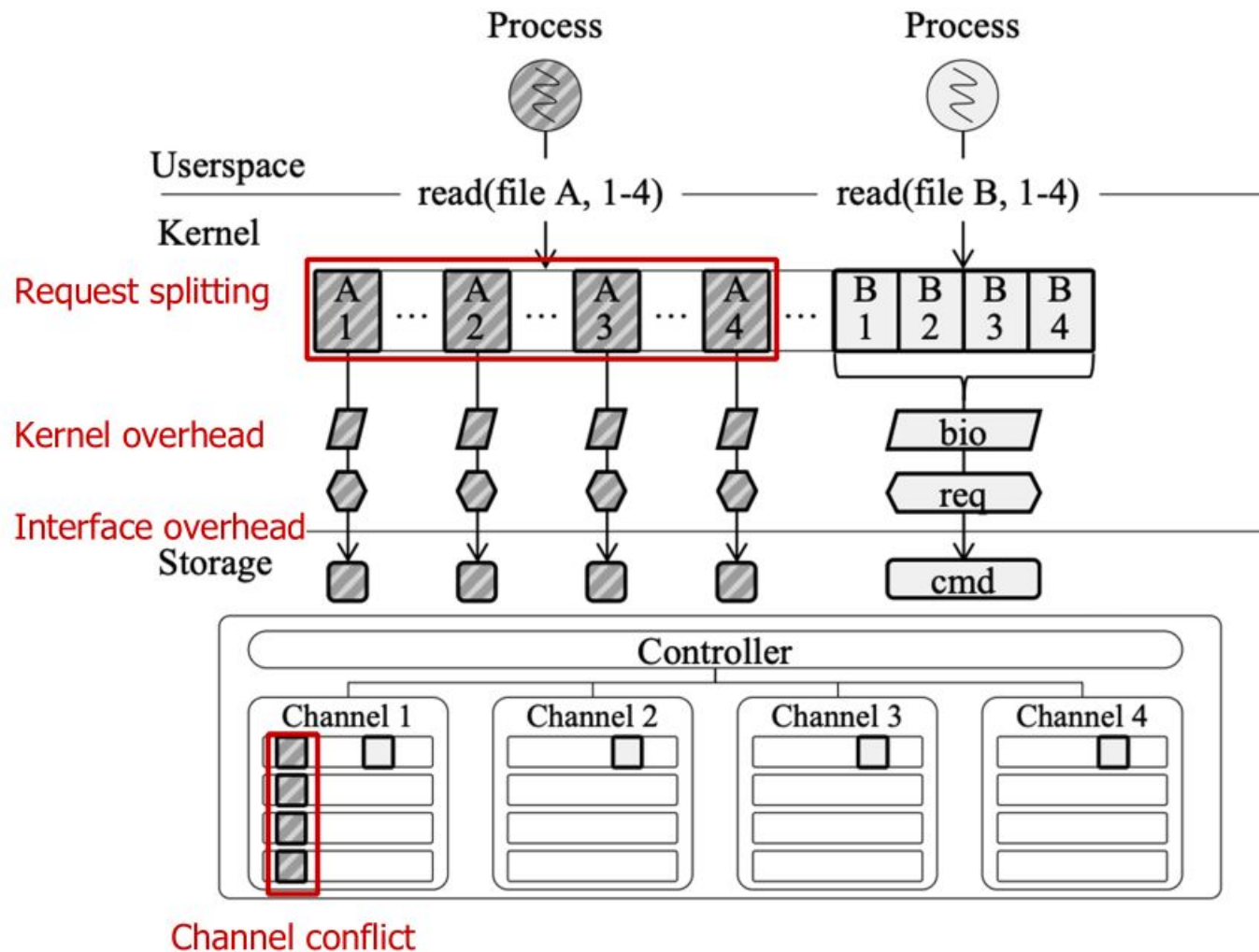
# 3. Background - Kernel I/O

## 기존의 분석 :

File 단편화로 인해 하나의 User I/O 요청이 여러 요청으로 분할됨

분할된 요청이 Kernel I/O path를 더 많이 통과

많은 Request가 SSD로 전달되어 Read overhead



# 4. Motivation

## 실험 환경

kernel I/O의 영향을 확인하기 위해 두가지 Disk 환경 (실제 NVMe SSD / Ramdisk)

각 Disk 환경에서 Queue Depth를 달리한 2개의 환경

총 4개의 환경에서 File Fragmentation에 따른 Read Time을 측정

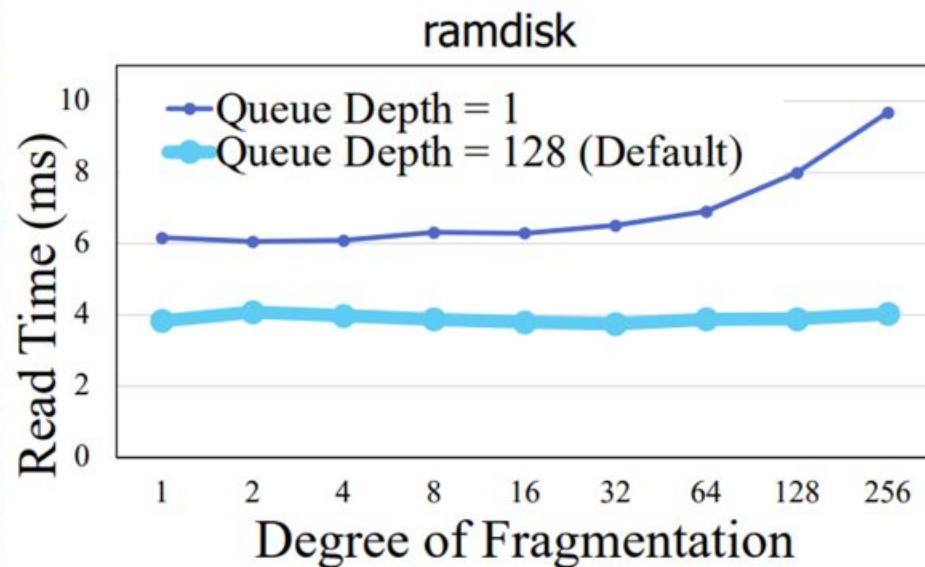
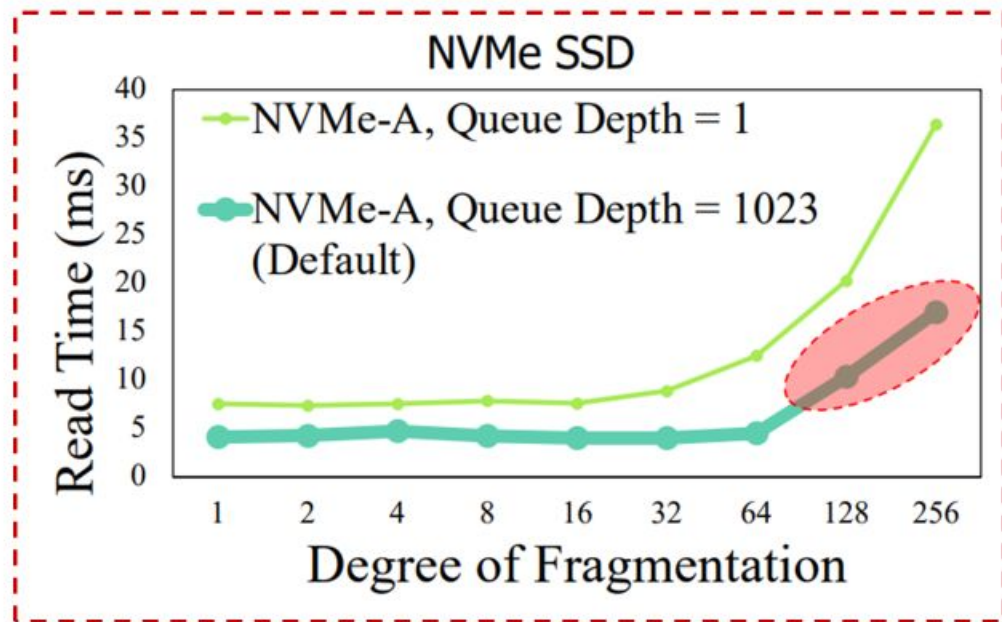
**Queue Depth:** SSD가 동시에 처리 할 I/O 요청 수

## 예상결과

기존 분석에 따른다면 File Fragmentation 정도가 심할수록,  
Read Time 또한 증가할 것이다.



## 4. Motivation

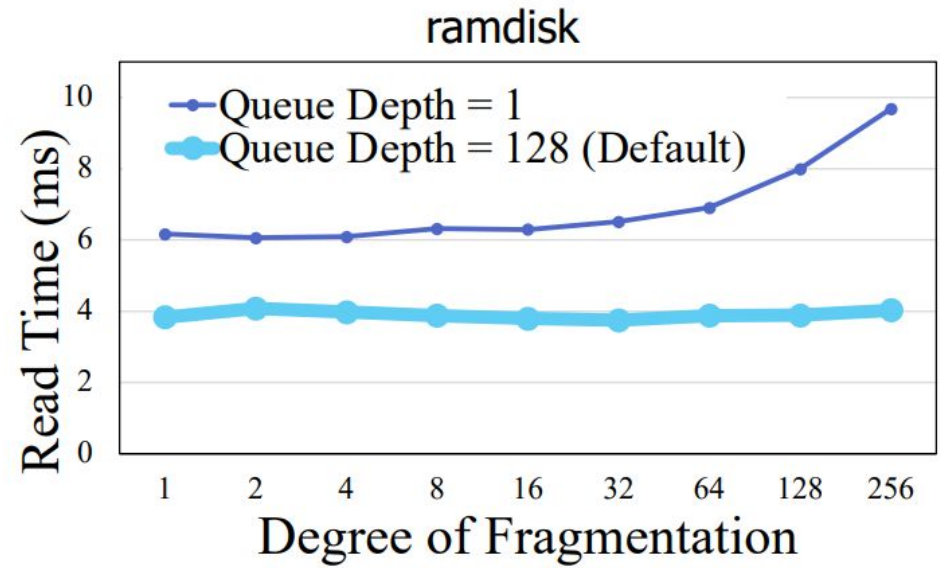
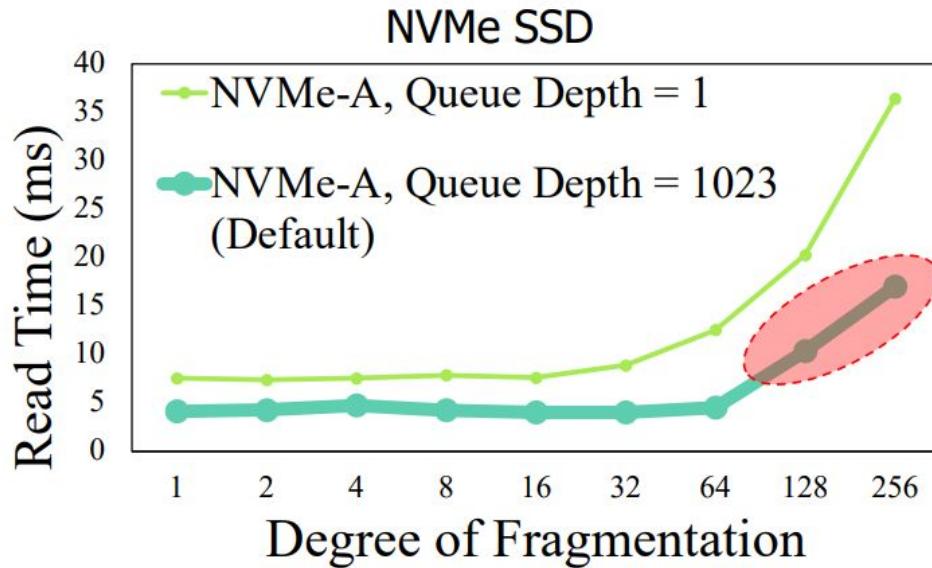


### 실제 NVMe SSD

실제 **SSD**의 내부 동작의 영향까지 포함

단편화가 심해지면, 임계점에서 Read Time이 증가하며, 이는 높은 Queue Depth에서도 발견된다.

## 4. Motivation : 단편화에 따른 읽기 지연 재현



### RAMDisk

**DISK 내부 동작의 영향을 최소화 → Kernel I/O Path의 영향만 분리**

Queue Depth=1일 때는 단편화에 따른 지연이 나타나지만, 높은 Queue Depth에서는 큰 성능 차이가 보이지 않는다.

## 4. Motivation : RamDisk 결과 분석

단편화가 심할수록, Request의 수와 생성시간이 증가. (fig. 1)

**Queue Depth = 1 일때**

Request 생성시간이 그대로 성능에 반영됨

**Queue Depth = 128 일때**

Request 생성시간이 길어져도, 높은 Queue Depth에 의해 그 영향이 작다. (fig. 2)

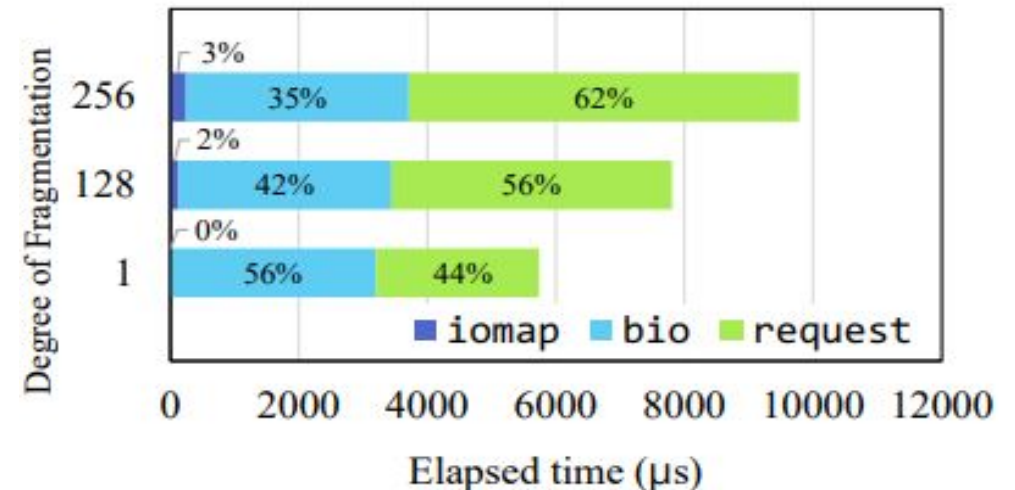


fig1: 단편화가 심해질수록, request 관련 자료구조 생성시간이 늘어나는것을 확인

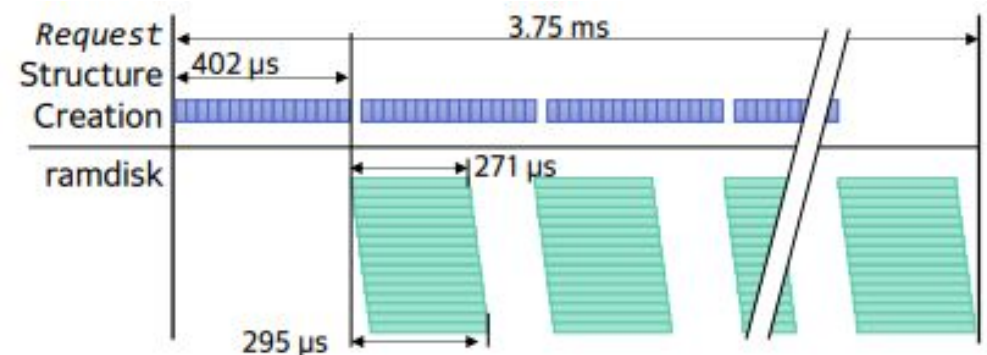
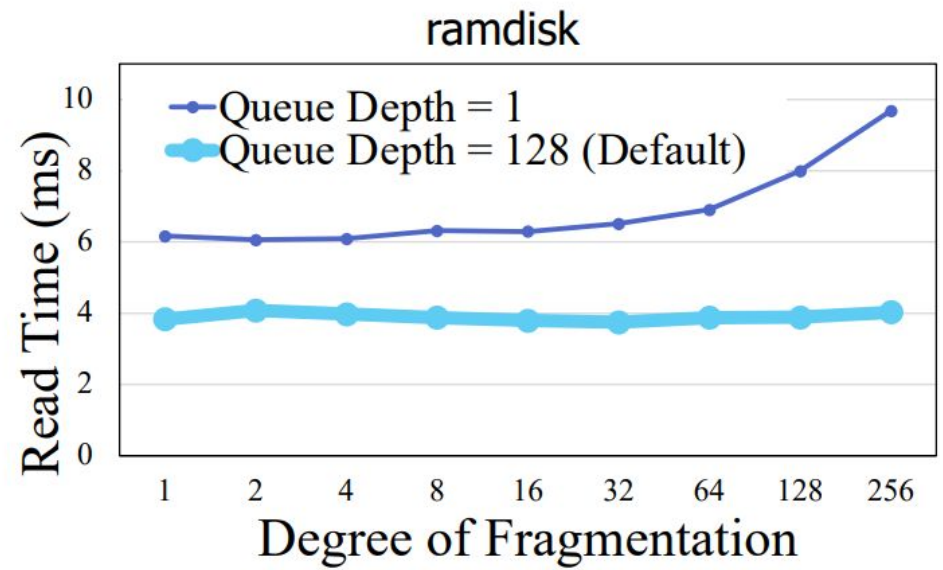
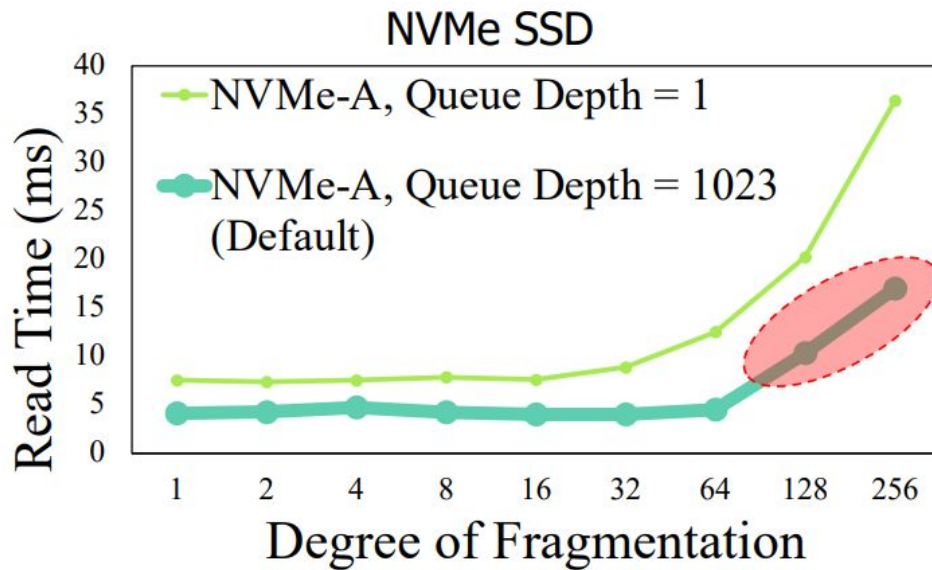


fig2: 스토리지 작업과 request 생성의 중첩으로 읽기 시간을 단축한다.

(Queue Depth: 128, Degree of Fragmentation: 128)

## 4. Motivation : 단편화에 따른 읽기 지연 재현



Ramdisk에선 NVMe SSD의 붉은 원 구간이 보이지 않는다.

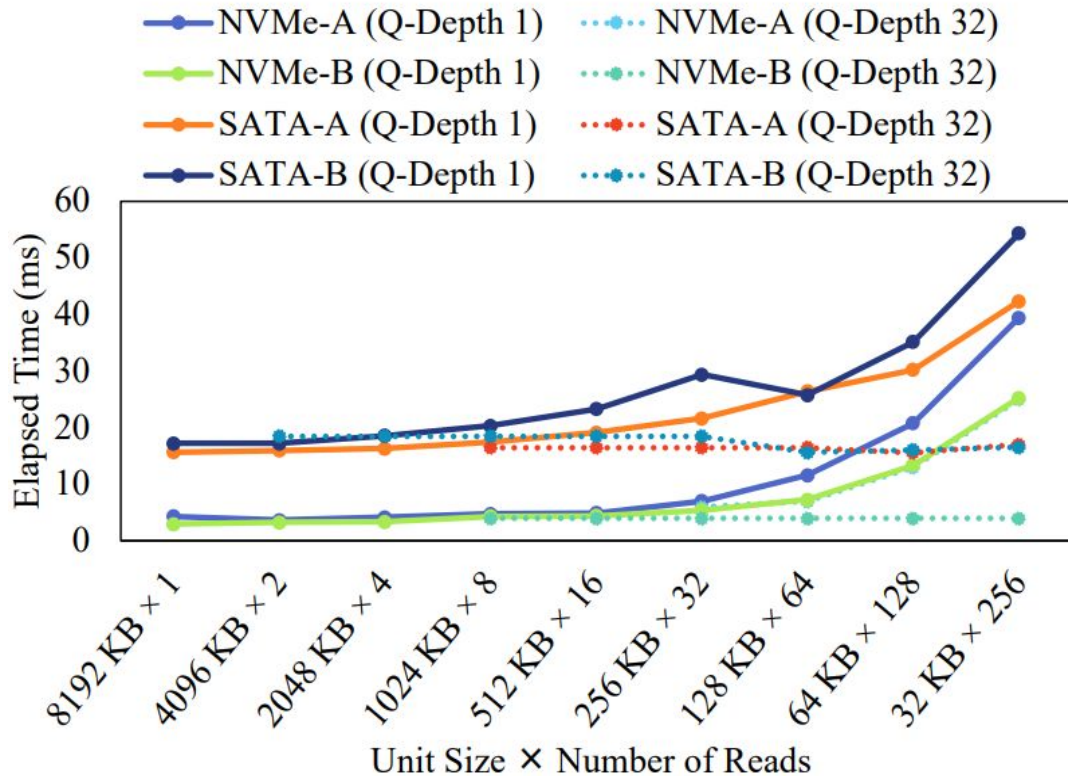
즉, 실제 SSD에서만 보이는 Read Time을 증가시키는 **Kernel I/O 외부의 문제가 존재**



# 5. Analysis of File Fragmentation

## 실제 SSD에서의 재검증

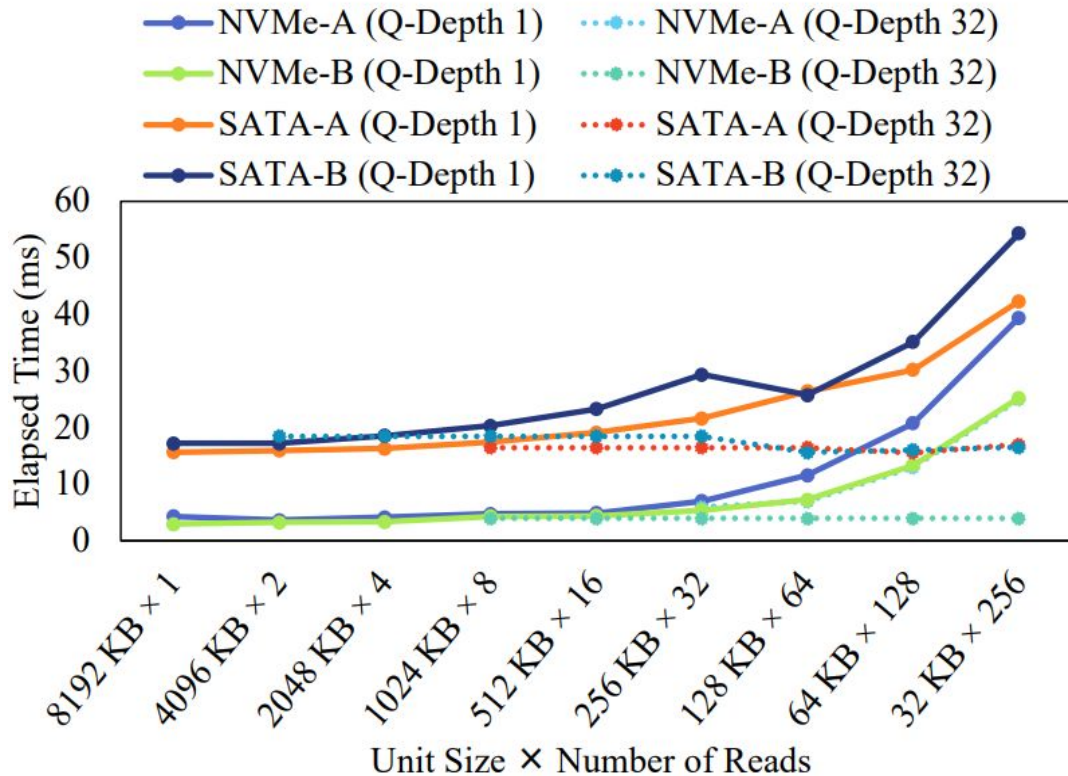
연속된 8 MB 공간을 여러 작은 Read 단위로 나누어 읽기



- **Raw Device 직접 접근**  
→ File System 등의 영향 배제
- **1 GB 영역 Sequential Write 후 측정**
- **한 번에 읽는 크기를 32 KB ~ 8 MB로 변화**  
→ 동일한 8 MB를 서로 다른 개수의 Request로 처리
- **Queue Depth = 1 / 32 비교**



# 5. Analysis of File Fragmentation

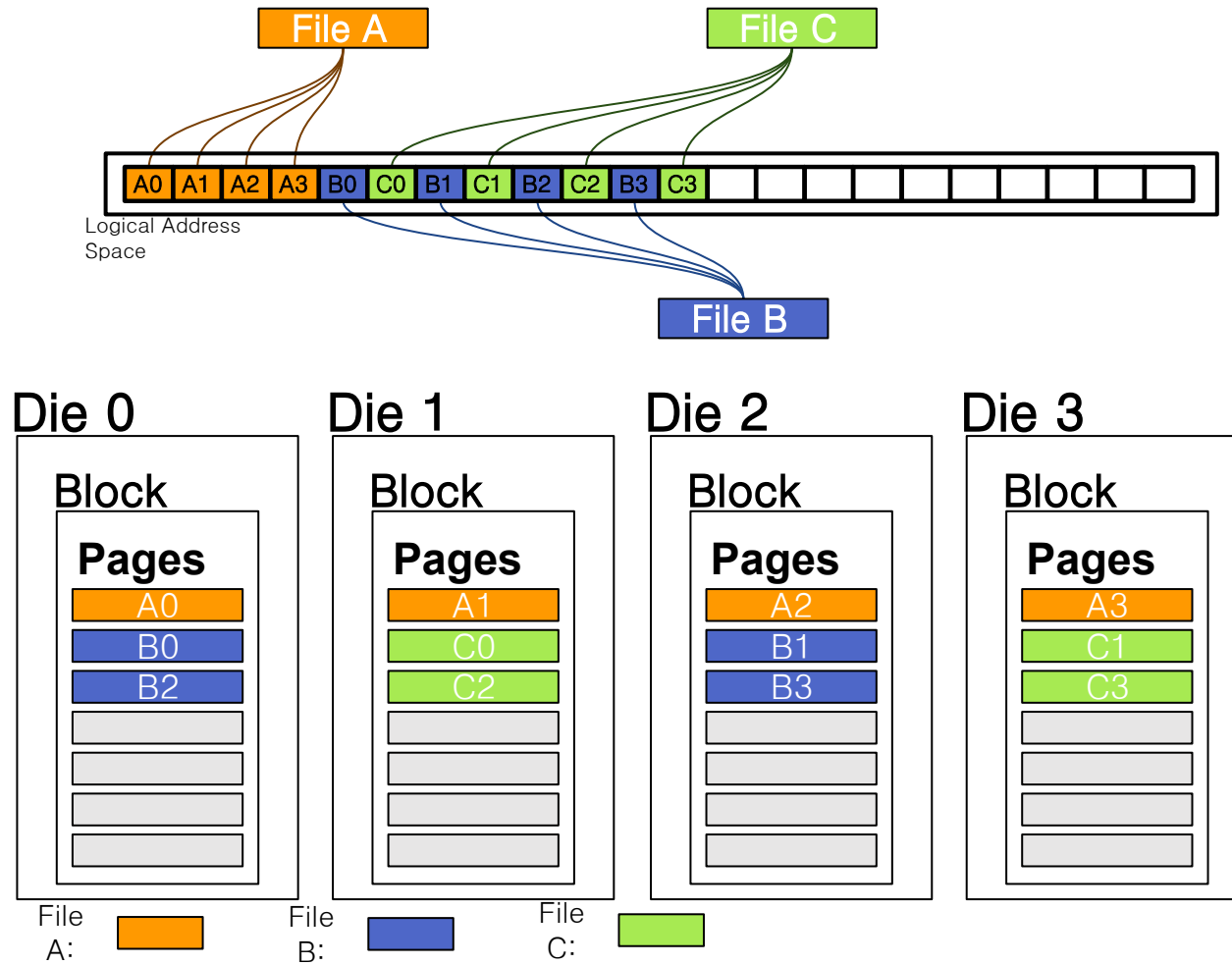


각 SSD 마다 세부적인 수치는 다르지만, 일관된 경향 존재

- **QD = 1**
  - Read Unit Size가 작아질수록 Request 수 증가
  - 전체 Read Time도 증가
- **QD = 32**
  - Request 수가 증가해도 실행시간 차이가 크지 않음
  - 여러 요청이 중첩 처리됨

이전 RAM-Disk 에서의 재현 실험과 비슷한 결과,  
기존에 지적하고 있던, **단편화로 인한 Request 수의 증가가 병렬성**을 가지고 있는 SSD에선 **주요한 문제가 아님**을 보임

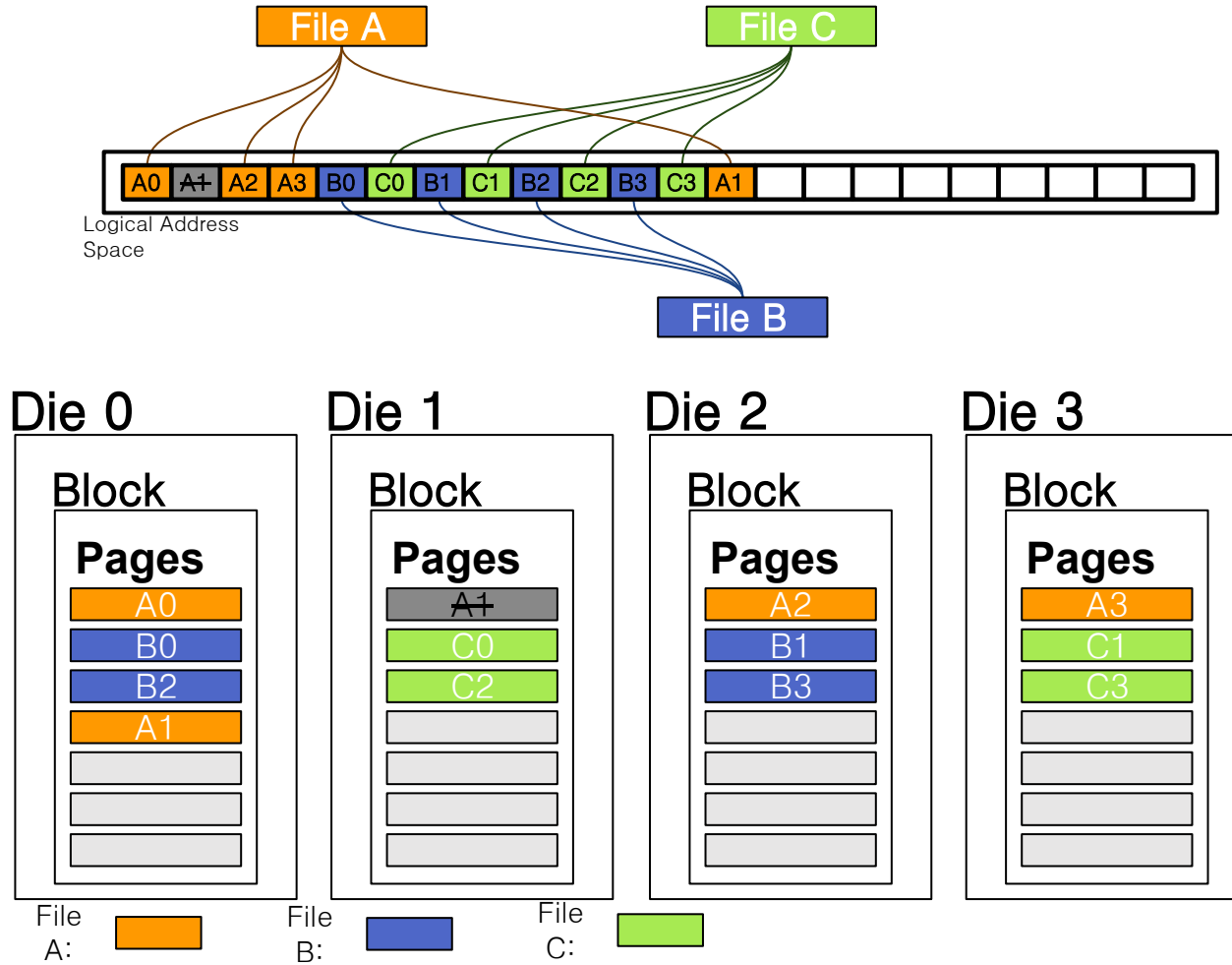
# 5. in the SSD: Die-level Collision



해당 그림과 같이 File B와 File C가 단편화된 경우, B와 C의 Page가 각 Die에 비균등하게 배치될 수 있다.

→ 이러한 배치에서는 B와 C를 읽을 때, 읽기 요청이 특정 Die에 집중되어 Die-level Collision이 증가한다.

# 5. in the SSD: Die-level Collision

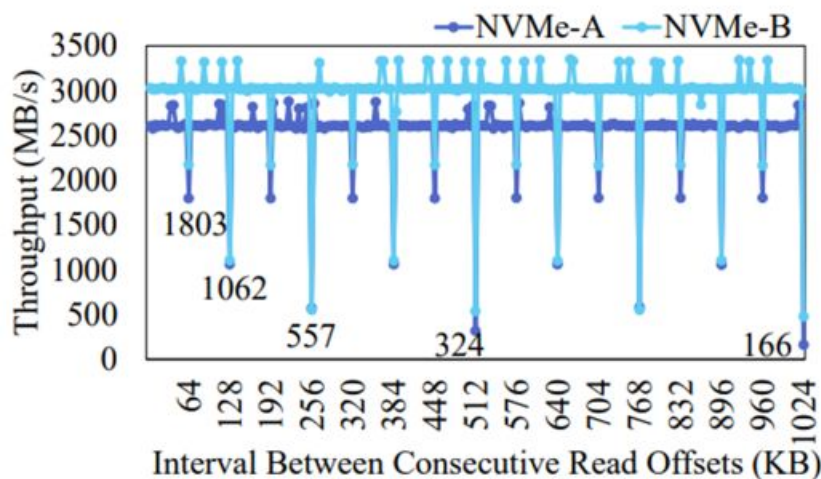


Overwrite 과정에서도 Page-to-Die 배치의 균등성이 깨진다.

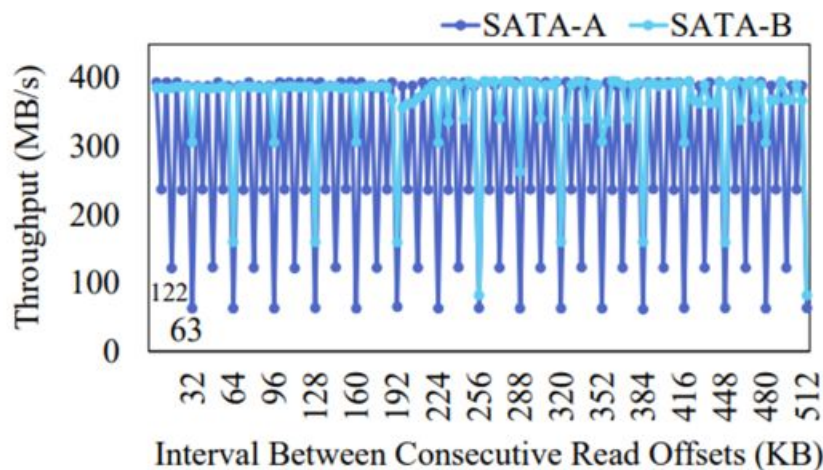
기존 Page는 **Invalid** 새 Page는 현재 Write 순서에 따라 **다른 Die**에 재배치

같은 파일의 블록들이 일부 Die에 **편중**  
Read 시 특정 Die에 요청 집중

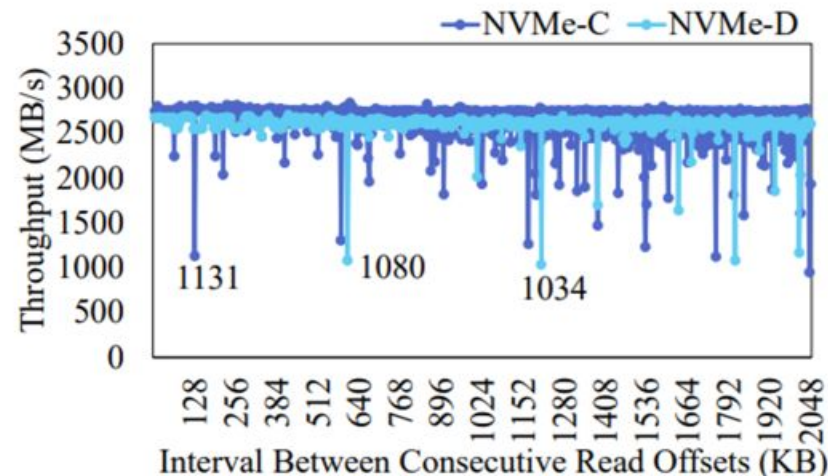
# 5. File Fragmentation Analysis - collision 검증 실험



(a) NVMe-A/B



(b) SATA



(c) NVMe-C/D

## 실험 방식

4KB 읽기 사이에 일정 용량의 interval을 두고,  
interval에 따른 Throughput을 측정

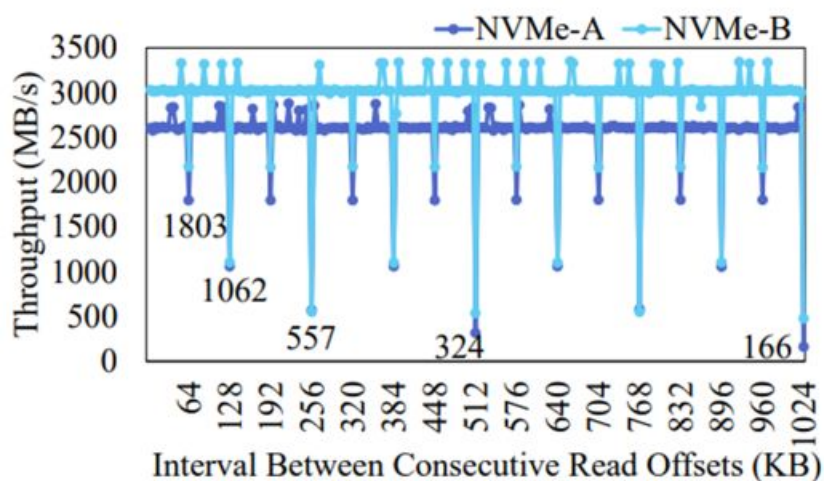
interval 예시



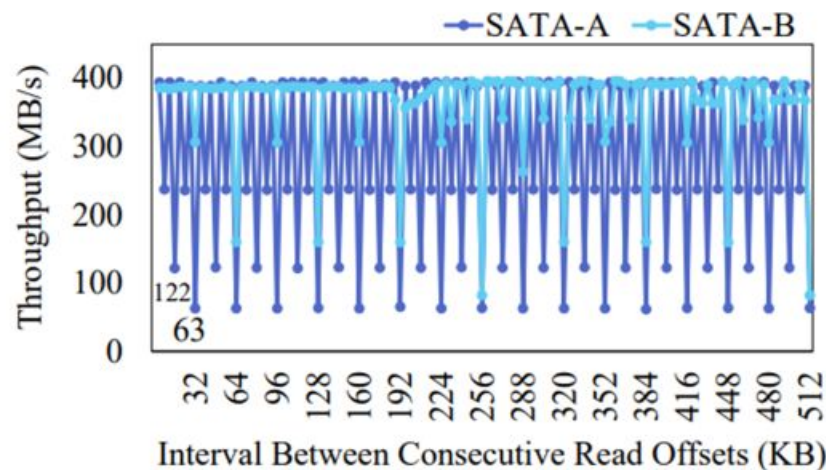
총 6개의 실험환경: NVMe-A, NVMe-B, NVMe-C, NVMe-D, SATA-A, SATA-B



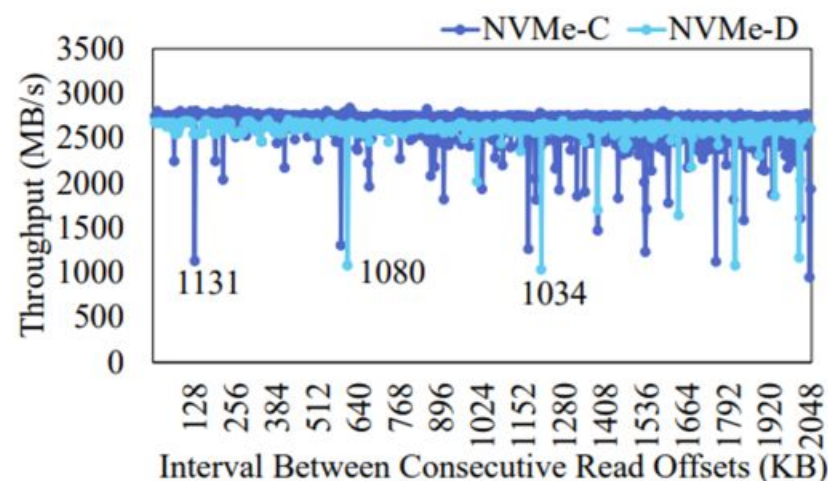
# 5. File Fragmentation Analysis - collision 검증 실험



(a) NVMe-A/B



(b) SATA



(c) NVMe-C/D

## 실험 결과

일정 간격마다 성능이 급감하는 지점이 발견된다.

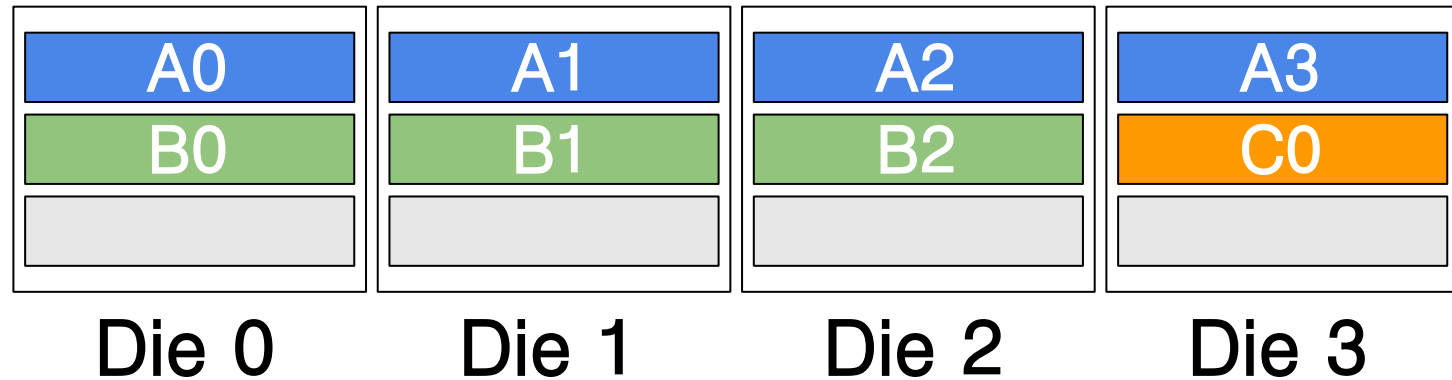
이때의 interval은 읽는 부분의 Die가 겹치는 지점으로 추정되어, SSD 병렬성의 영향이 보이게 된다.



## 6. Our Approach : Die-level Collision 해결책

NVMe 명령어에, 배치할 **die**  
정보까지 함께 보낸다

Initial State



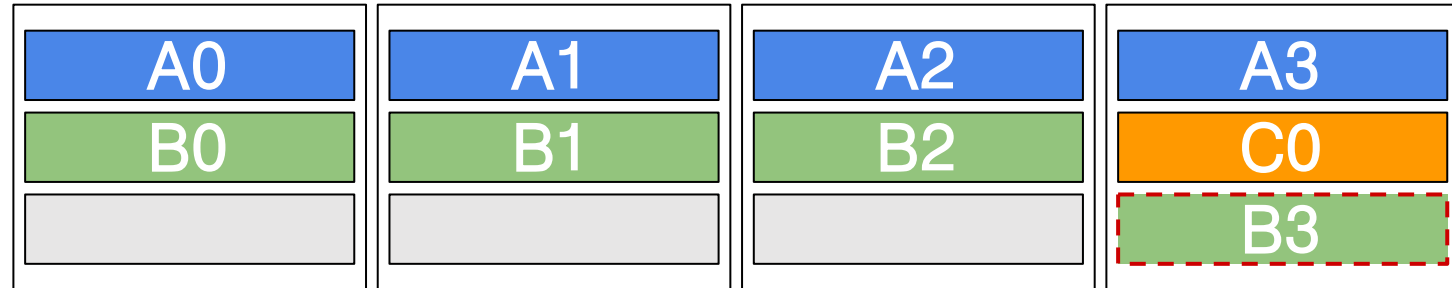
## 6. Our Approach : Die-level Collision 해결책

NVMe 명령어 전송에, 배치할 **die** 정보까지 함께 보낸다

- **Append**

이전 File Block이 위치한 Die의  
다음 Die

File B Append <B3>



# 6. Our Approach : Die-level Collision 해결책

NVMe 명령어 전송에,  
배치할 **die** 정보까지 함께 보낸다

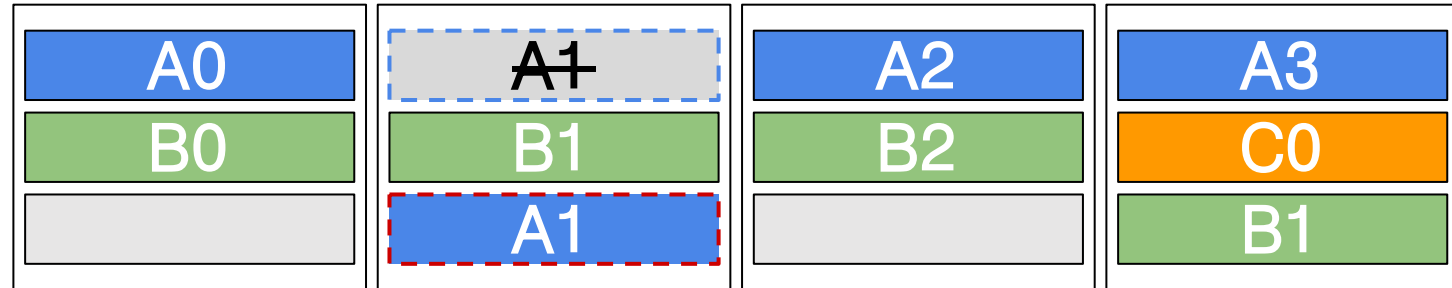
- **Append**

File의 이전 Page의 다음 Die

- **Overwrite**

삭제한 기존 Page와 동일 Die

File A Overwrite <A1>



## 6. Our Approach

NVMe 명령어 전송에,  
배치할 **die** 정보까지 함께 보내기

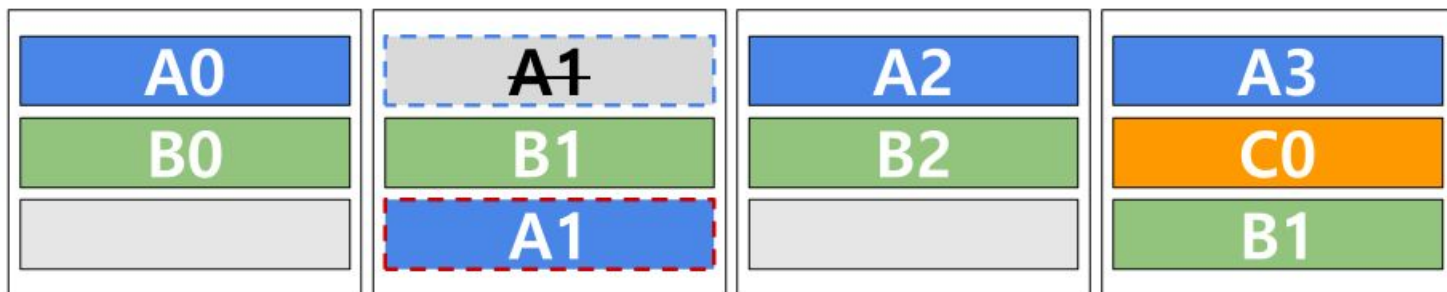
- **Append**

File의 이전 Page의 다음 Die

- **Overwrite**

삭제한 기존 Page와 동일 Die

File A Overwrite <A1>



**Append / Overwrite 시에도 File의 Die간 균등을 유지**



# 7. Evaluation : Experiment Setup

- 실제 상용되는 SSD의 실제 firmware 수정은 불가능

→ 논문의 제안과 동일한 page-to-die mapping이 만드는 write pattern을 인위적으로 만들어 read 성능 측정

→ Metadata / Journal Write의 영향을 최소화하도록 별도 처리

- NVMe-A, NVMe-B, SATA-A, SATA-B 4개의 SSD 환경에서 실험

# 7. Evaluation

## Fragmentation 실험

- 초기 상태의 SSD에서 8MB 파일 생성
- 256개의 fragment를 append
  - die allocation granularity = 32KB
  - NVMe :  $32\text{KB} \times 256 = 8\text{MB}$
- SATA : granularity 차이로 256회만으로 8MB가 되지 않아 마지막 fragment 크기 조정

→ 파일 앞 부분만 fragmentation 발생

# 7. Evaluation

## Fragmentation 실험

1. Target file에 fragment 1개 작성
  2. Dummy file에 SSD의 남은 stripe 공간만큼 write
  3. 다음 fragment를 target filie에 작성
  4. 위 과정을 반복
- Target file의 fragment가 동일한 die에 배치

Target File

|    |       |    |       |
|----|-------|----|-------|
| A0 | dummy | A1 | dummy |
|----|-------|----|-------|

SSD

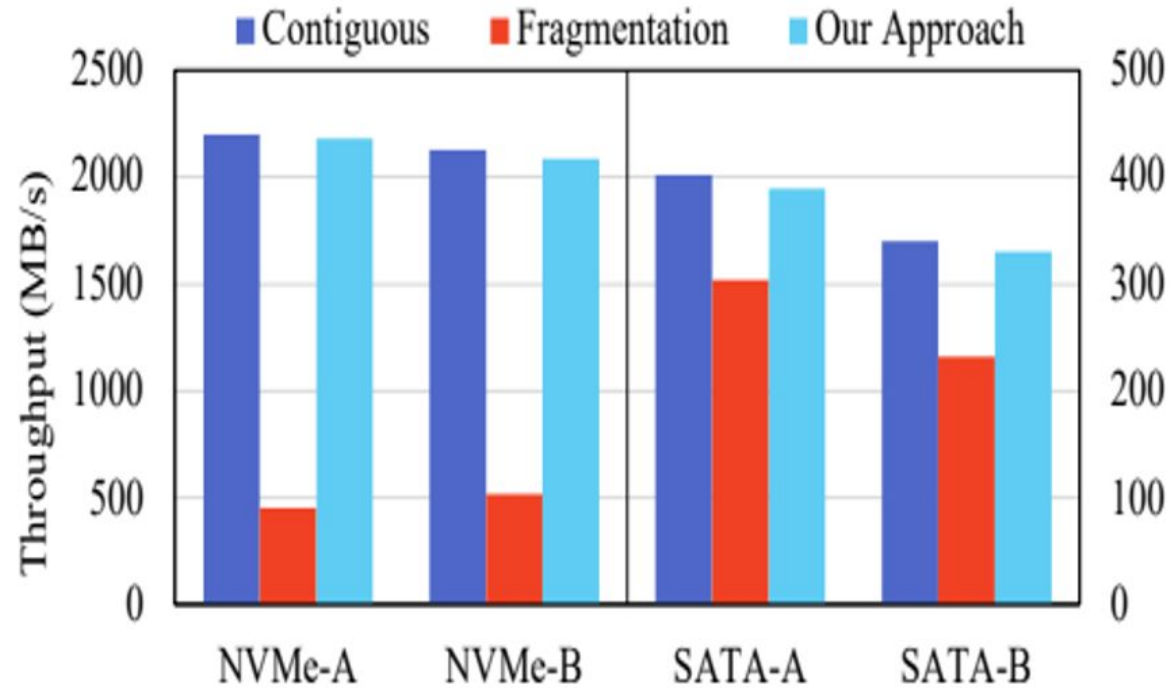
|          |       |       |       |
|----------|-------|-------|-------|
| A0<br>A1 |       |       |       |
| Die 1    | Die 2 | Die 3 | Die 4 |

# 7. Evaluation : Validation of Our Approach

Our Approach 실험

1. Target file에 fragment 1개 작성
2. Dummy file에 SSD의 stripe size만큼 garbage data 작성
3. Target file에 다음 fragment 작성
4. 위 과정 반복 → 각 fragment가 이전 fragment의 다음 die에 배치되도록 유도

# 7. Evaluation : Validation of Our Approach



(a) Append Write

Figure 11: Read performance of four kinds of SSDs for contiguous files, fragmented files, and fragmented files under our proposed approach, respectively.

NVMe SSD : Fragmentation 실험 결과

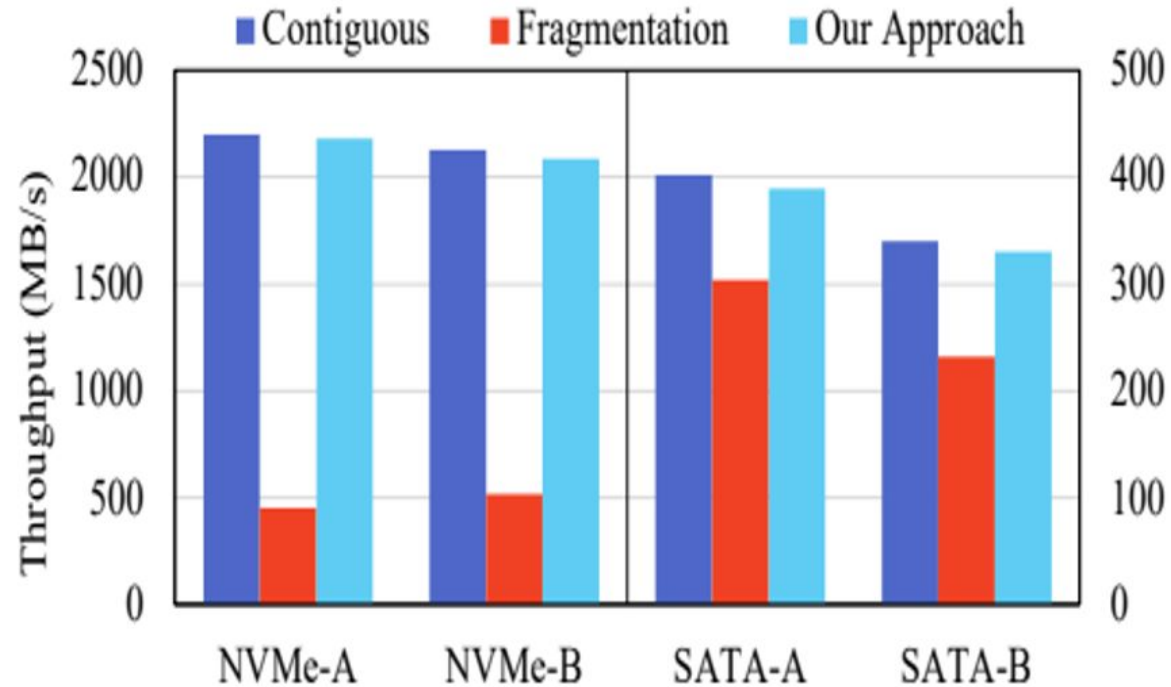
- NVMe-A : Read 성능 79% 감소
- NVMe-B : Read 성능 76% 감소

→ NVMe-A가 더 많은 die를 보유하여 성능 저하가 더 크게 나타남

- Fragment가 하나의 die에 집중되어 대부분의 read에서 die-level collision 발생
- 논문 방법을 적용시, contiguous 대비 약 2% 성능 저하



# 7. Evaluation : Validation of Our Approach



(a) Append Write

Figure 11: Read performance of four kinds of SSDs for contiguous files, fragmented files, and fragmented files under our proposed approach, respectively.

SATA SSD : Fragmentation 실험 결과

- Die allocation granularity가 작아 파일 일부만 fragmentation → NVMe보다 성능 저하 작음
- SATA-A : Read 성능 27% 감소
- SATA-B : Read 성능 16% 감소
- 논문의 방법 적용시, 최대 1.2% 성능 저하

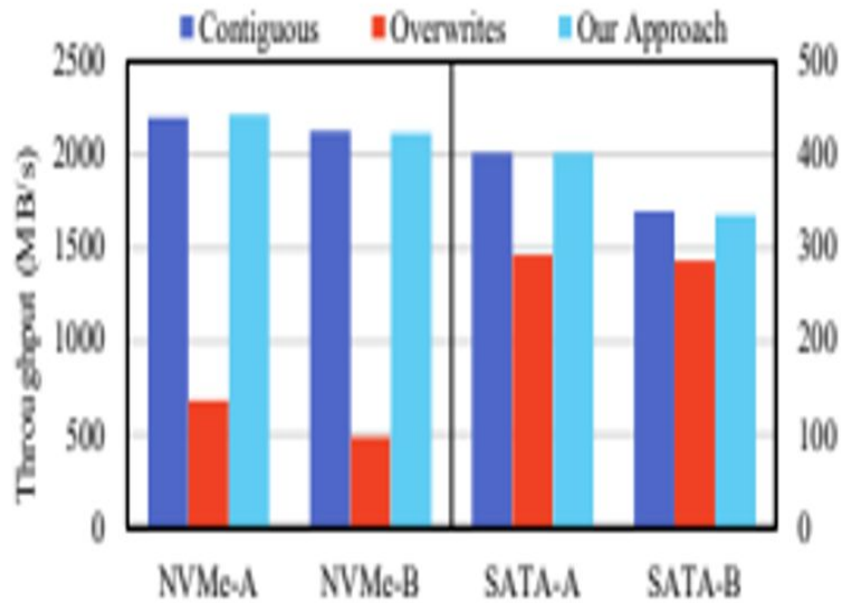
# 7. Evaluation : Validation of Our Approach

Overwrite 실험 : Overwrite로 인한 page-to-die mapping misalignment 검증

1. 8MB 파일 생성
2. 파일 전체에 32KB Overwrite 256회 수행
3. Overwrite 사이마다 Stripe size - Die allocation Granularity만큼 dummy file에 random data 작성
4. 파일 read

→ Overwrite된 새 page들이 하나의 die에 몰리기 때문에 die-level collision 발생, FS 수준에서는 data block이 여전히 contiguous 상태

# 7. Evaluation : Validation of Our Approach



(b) Overwrite

Figure 11: Read performance of four kinds of SSDs for contiguous files, fragmented files, and fragmented files under our proposed approach, respectively.

## Overwrite 실험 결과

- Overwrite 만으로도 Read 성능이 약 1/4 수준으로 감소  
→ Page-to-die mapping이 깨지며 die-level collision 발생
- Fragmentation과 유사한 수준의 성능 저하 확인
- 논문의 방법 적용 시 평균 성능 저하 1%로 감소

# 7. Evaluation : Effectiveness for Application Workloads

실제 Application 실험 환경

- Host side : EXT4 FS + Linux NVMe driver에 제안 방법 구현
- SSD side : NVMeVirt에 제안 방식 구현
- NVMeVirt 설정
  - Die allocation granularity : 32KB
  - Stripe size : 256KB

# 7. Evaluation : Effectiveness for Application Workloads

실험 환경

Random Disturbance

- Target file 사이에 random 크기의 dummy write 삽입

→ File block이 하나의 die에 집중되지 않고 여러 die에 분산

# 7. Evaluation : Effectiveness for Application Workloads

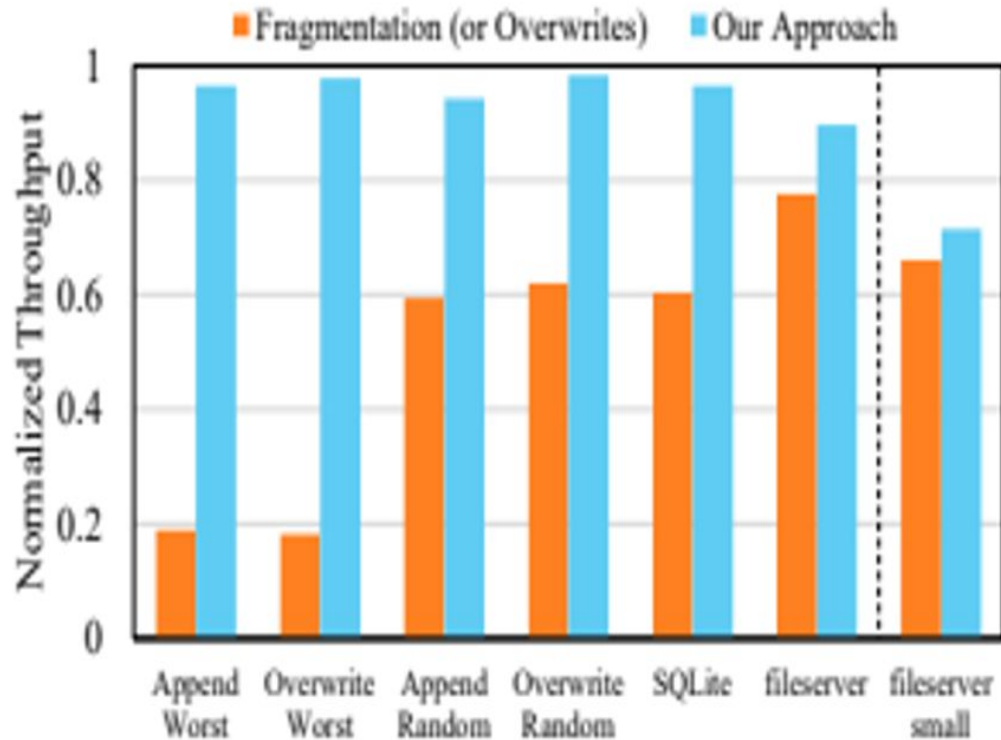


Figure 12: Normalized read throughput of applications executed with the implementation of our approach relative to that with ideal file block and flash page placement.

## Worst-case 결과

- 기존 방식
  - Append Worst : 정상 성능의 19%
  - Overwrite Worst : 정상 성능의 18.2%
- Our Approach
  - Append Worst : 3.5%의 성능 저하
  - Overwrite Worst : 2.3%의 성능 저하

→ 모든 file block이 하나의 die에 집중되면 Die-level collision 증가로 인해 Read 성능 크게 저하



# 7. Evaluation : Effectiveness for Application Workloads

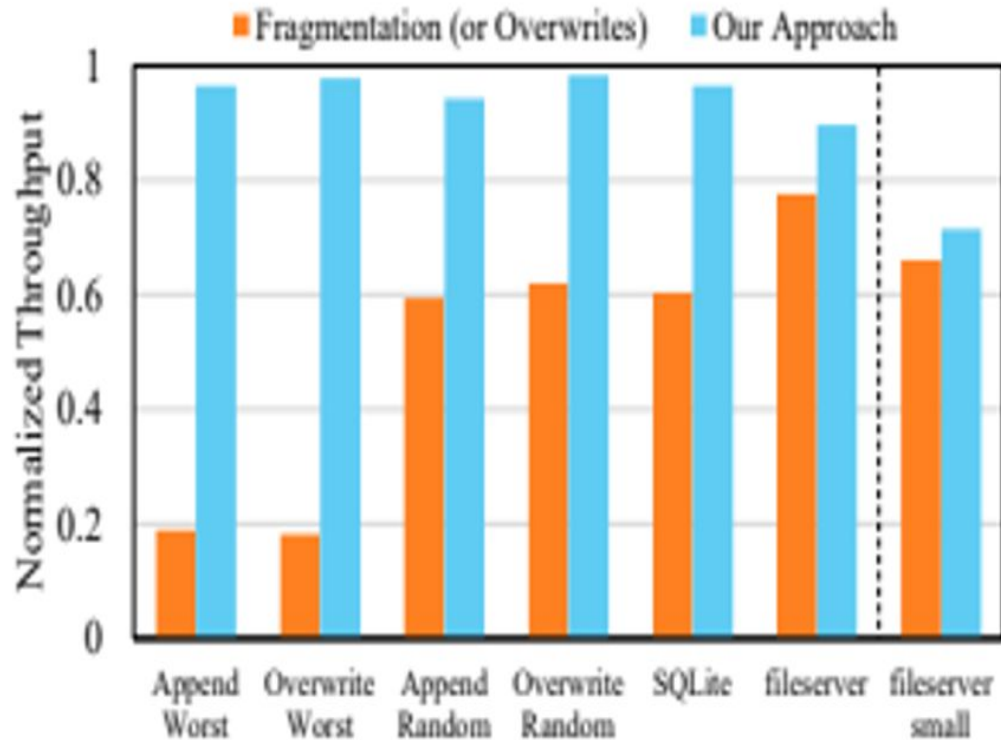


Figure 12: Normalized read throughput of applications executed with the implementation of our approach relative to that with ideal file block and flash page placement.

## Random Disturbance 결과

- 기존 방식
  - Append Random : 성능 40.6% 감소
  - Overwrite Random : 성능 38% 감소
- Our Approach
  - Append Random : 성능 5.8% 감소
  - Overwrite Random : 성능 1.6% 감소

→ 모든 file block이 하나의 die에 몰리지 않아도 적절한 page-to-die mapping을 유지하면 성능 저하 크게 감소

# 7. Evaluation : Effectiveness for Application Workloads

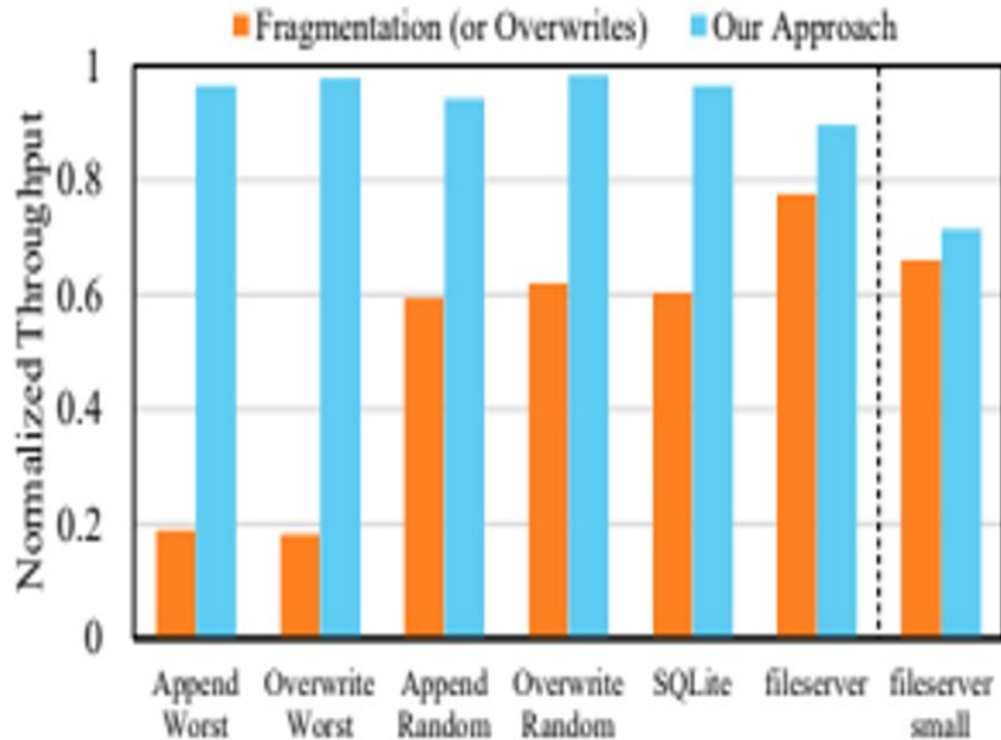


Figure 12: Normalized read throughput of applications executed with the implementation of our approach relative to that with ideal file block and flash page placement.

## SQLite 결과

- 10,000 records X 16KB의 SQLite DB 생성
- 동시에 dummy file에 100KB append 반복

→ DB file fragmentation 발생, DoF = 5,005

- 기존 방식
  - 전체 records SELECT, 약 40%의 성능 감소
- Our Approach
  - DB file block을 consecutive dies에 배치

→ SELECT 성능 1.6배 향상, 약 3.5%의 성능 감소

# 7. Evaluation : Effectiveness for Application Workloads

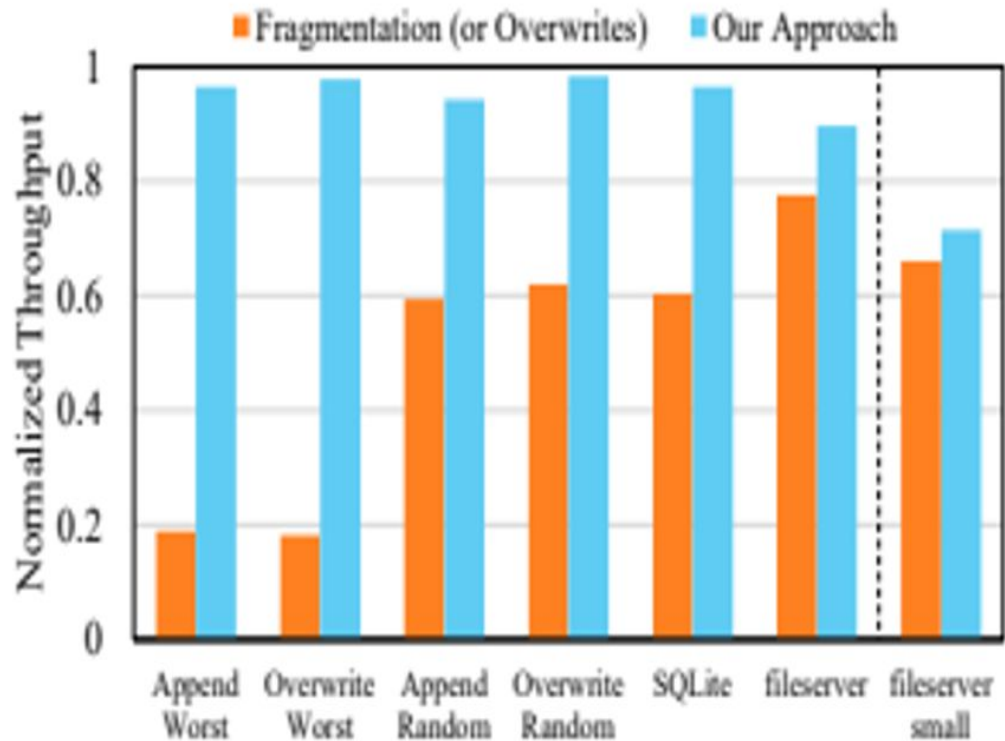


Figure 12: Normalized read throughput of applications executed with the implementation of our approach relative to that with ideal file block and flash page placement.

Fileserver 결과

File set : 10,000 files, 평균 크기 128KB

Fragmentation 유도

1. 10,000개의 128KB 파일을 미리 preallocate
2. 10개 thread가 random file을 선택해 32KB append
3. 1분 동안 실행 (File creation / deletion은 제외)

# 7. Evaluation : Effectiveness for Application Workloads

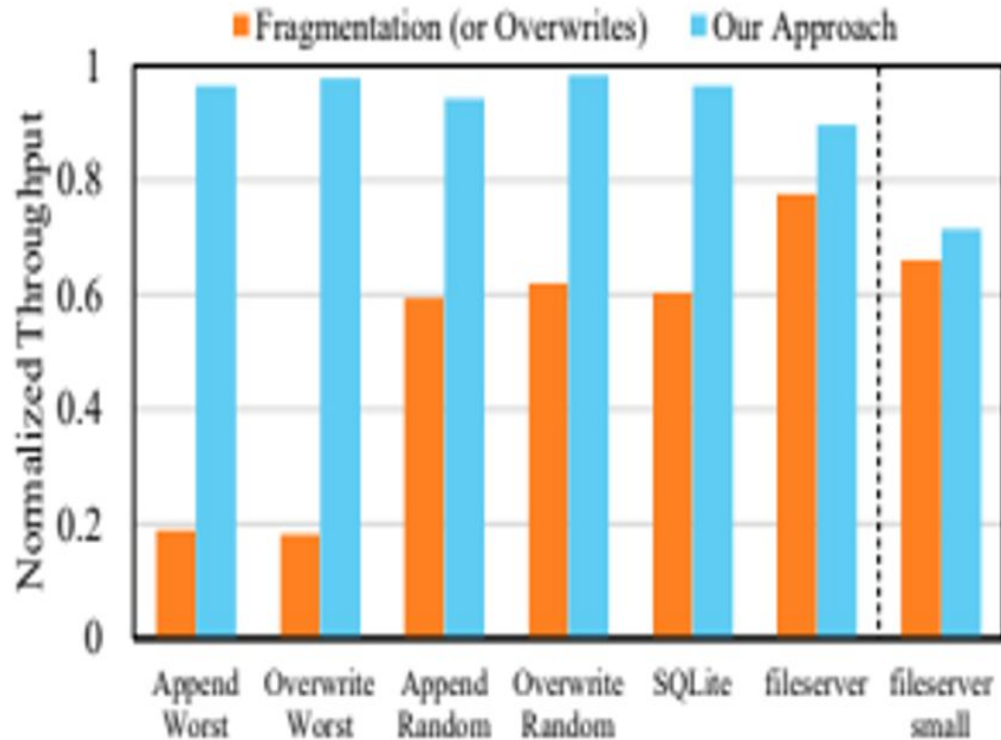


Figure 12: Normalized read throughput of applications executed with the implementation of our approach relative to that with ideal file block and flash page placement.

## Fileserver 실험 결과

- 기존 방식 : Read 성능 약 20% 감소
- Worst-case보다 성능 저하가 작게 나타남
  - 여러 thread가 동시에 read
  - → Outstanding command 증가
  - → Die-level parallelism이 어느 정도 유지
  - Fragmentation의 성능 영향 감소
- Our Approach : sequential read 성능을 이상적인 경우의 93%까지 회복

# 7. Evaluation : Limitations of Fileserver Experiment

- Append 크기를 32KB → 16KB로 감소
- SSD의 Flash Page 크기 = 32KB
- 서로 다른 16KB write 2개가 하나의 Flash Page에 함께 저장 가능 → 하나의 File이 더 많은 Flash Page에 걸쳐 저장 → 파일 read 시 읽어야 할 Flash Page 수 증가 → Read 성능 저하
- 결과적으로 성능 향상은 8.2%에 그침
- 즉, 여러 파일의 작은 write가 하나의 Flash Page에 섞이는 page-level fragmentation은 해결하지 못 함

# 7. Evaluation : Future Work

- 현재 방법은 file block의 파일의 잘못된 Page-to-die 배치 해결에 초점
- 여러 file의 작은 write가 하나의 flash page에 섞이는 문제는 미해결
- Flash Page-level fragmentation까지 고려한 새로운 page allocation strategy 필요



# 8. Conclusion

- SSD에서도 File Fragmentation은 Read 성능 저하를 유발
- 주요 원인 : 잘못된 Page-to-die 배치 → Die-level Collision 증가
  - Fragmentation 뿐만 아니라 Overwrite에서도 발생
- NVMe Command Extension + Page-to-Die Mapping Scheme 제안
  - 파일 순서에 따라 Page를 연속적인 die에 배치
- Defragmentation 없이 Fragmentation / Overwrite로 인한 성능 저하를 수 % 수준까지 억제

# Thank you